

Algorithmes de graphe

GraphX / GraphFrames avec PySpark

Michel Cadennes – 2026

Objectifs pédagogiques

- * Modéliser un problème réel sous forme de graphe (sommets, arêtes, propriétés)
- * Créer et manipuler un graphe avec GraphFrames en PySpark
- * Appliquer les algorithmes fondamentaux : BFS, composantes connexes, triangle counting
- * Implémenter et interpréter PageRank sur un graphe distribué
- * Identifier les cas d'usage concrets des graphes en IA et en data science

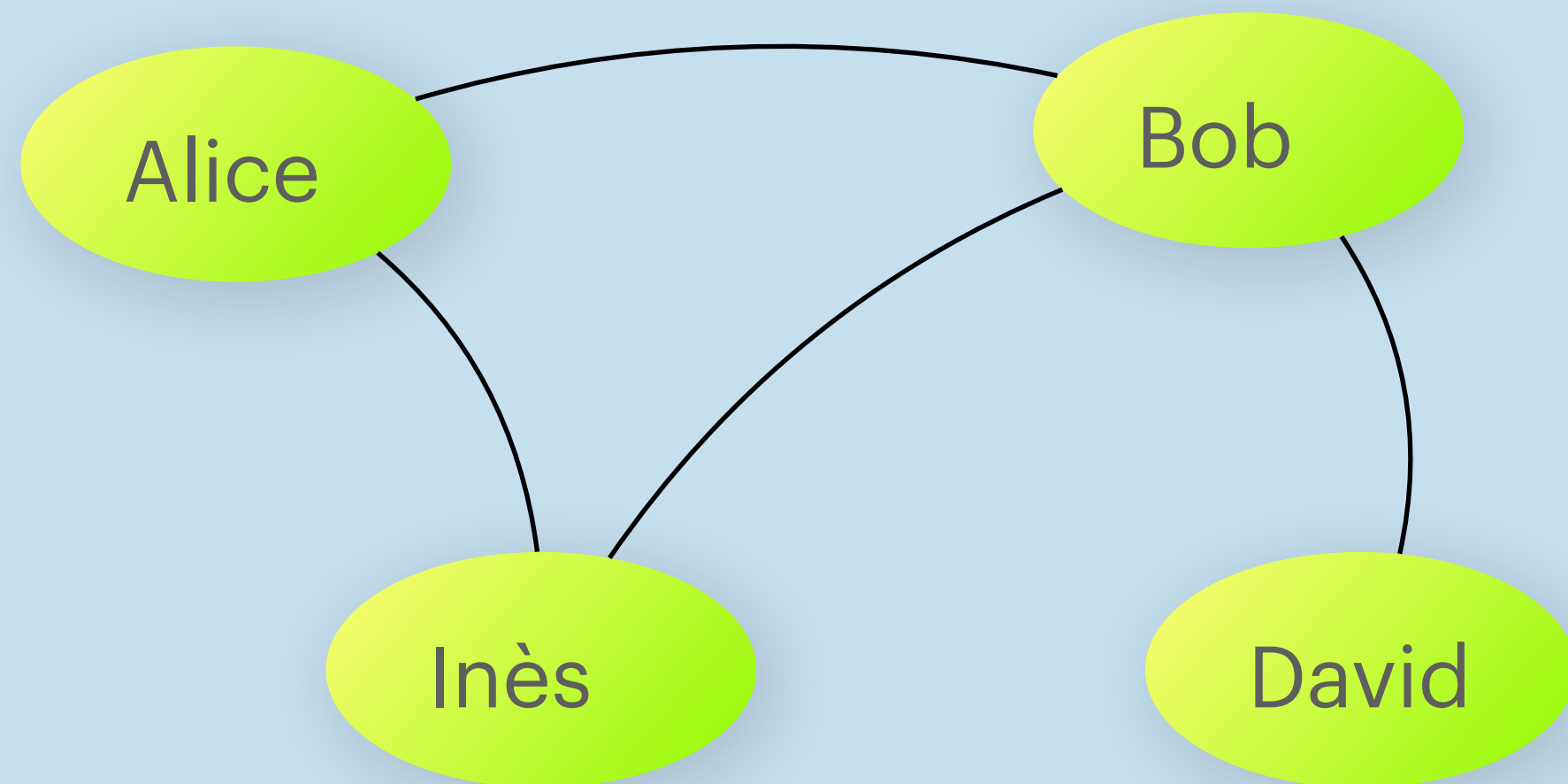
Introduction à la théorie des graphes

Graphe $G = (V, E)$

Définitions fondamentales

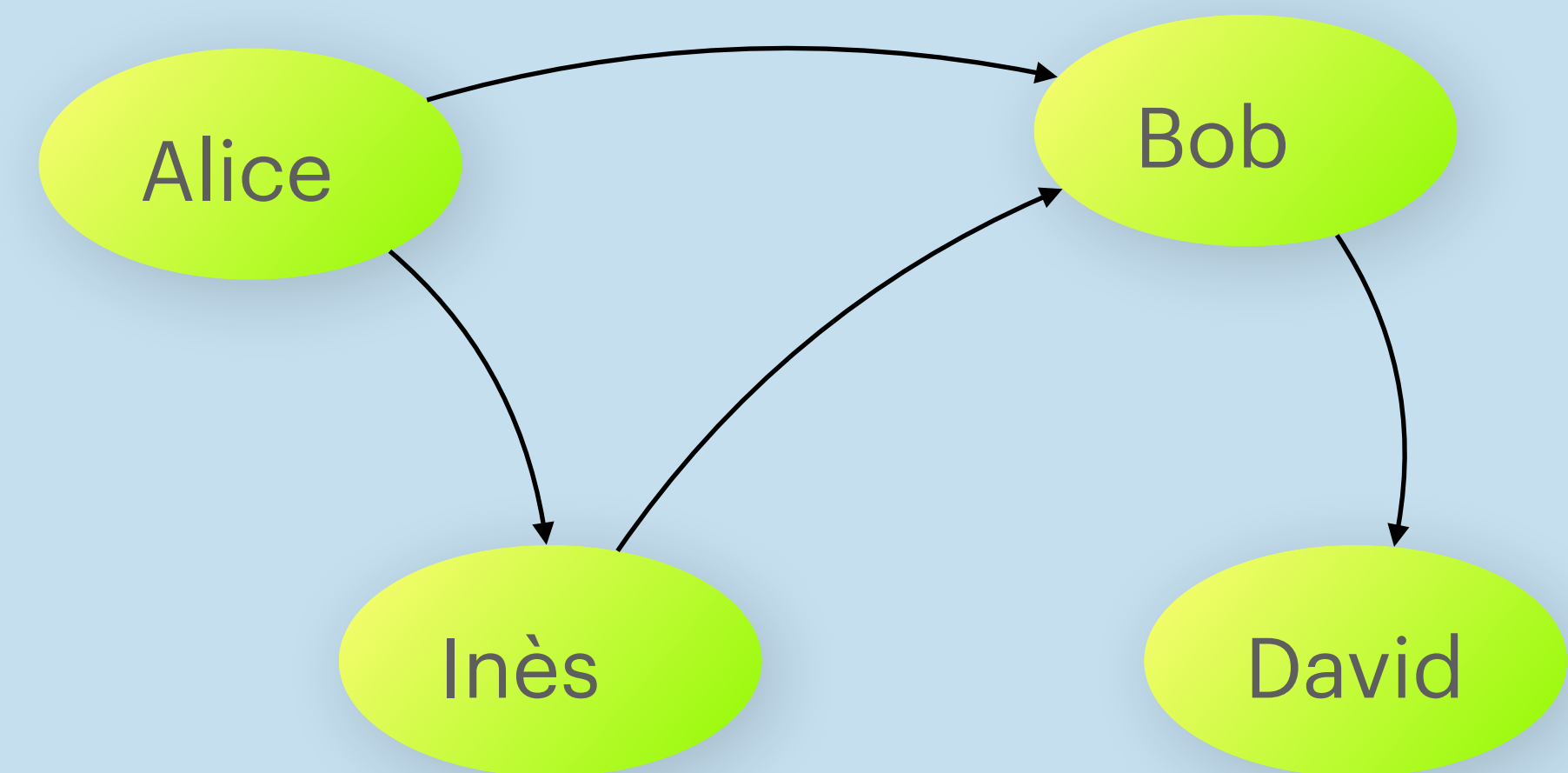
Sommets (V = Vertices) + Arêtes (E = Edges) — orienté ou non, pondéré ou non

Graphe non orienté



Arêtes bidirectionnelles

Graphe orienté (digraphe)



Arêtes avec direction source → destination

Types de graphes

Propriété	Description
Orienté / Non orienté	Les arêtes ont-elles un sens ? Twitter (orienté) vs Facebook (non orienté)
Pondéré	Les arêtes ont-elles un poids ? Distance routière, force d'une relation
Connexe	Peut-on atteindre tout sommet depuis n'importe quel autre ?
Cyclique	Existe-t-il des chemins qui reviennent au point de départ ?
Biparti	Sommets en 2 groupes, arêtes uniquement entre groupes (ex : Utilisateurs ↔ Produits)

Vocabulaire clé et cas d'usage en IA

Vocabulaire essentiel

Degré	Nb d'arêtes connectées à un sommet (entrant / sortant)
Chemin	Suite de sommets reliés par des arêtes
Plus court chemin	Chemin minimisant le nb d'arêtes ou leur poids
Diamètre	Longueur du plus long des plus courts chemins
Triangle	3 sommets tous reliés deux à deux
Composante connexe	Sous-graphe connexe maximal (îlot isolé)
Communauté	Groupe de sommets très interconnectés

Cas d'usage en IA et data science

Réseaux sociaux	→ PageRank, communautés
Moteurs de recherche	→ PageRank (Google)
Détection de fraude	→ Composantes connexes, cycles
Recommandation	→ Propagation de labels
Knowledge Graph	→ Recherche de chemins, inférence
NLP — TextRank	→ Résumé automatique
Logistique	→ Plus court chemin, flot max

GraphX vs GraphFrames

GraphX vs GraphFrames

En PySpark : GraphFrames est le seul choix — GraphX est réservé à Scala/Java

GraphX

- RDD-based (bas niveau)
- Scala et Java uniquement
- API verbeuse mais très performante
- Natif dans Spark — aucune dépendance
- Accès direct aux opérateurs Pregel

GraphFrames PySpark

- DataFrame-based (haut niveau)
- Python, Scala, Java, R
- API concise et expressive
- Package externe (graphframes)
- Donne accès à la plupart des algos GraphX

Installation de GraphFrames

Via spark-submit

```
spark-submit --packages graphframes:graphframes:0.8.3-spark3.5-s_2.12 mon_script.py
```

Dans le code Python

```
spark = SparkSession.builder\  
    .config("spark.jars.packages","graphframes:graphframes:0.8.3-spark3.5-s_2.12")\  
    .getOrCreate()  
from graphframes import GraphFrame
```

Création d'un GraphFrame

Structure d'un GraphFrame

- * **vertices**
(colonne id obligatoire)
- * **edges**
(colonnes src et dst obligatoires)

DataFrame des SOMMETS

```
vertices = spark.createDataFrame([
    ("alice", "Alice", "Ingénieure", 35),
    ("bob", "Bob", "Analyst", 28),
    ("claire", "Claire", "Chercheuse", 42),
    ("david", "David", "Ingénieur", 31),
    ("emma", "Emma", "Étudiante", 24),
],
["id","nom","metier","age"])
```

DataFrame des ARÊTES

```
edges = spark.createDataFrame([
    ("alice","bob", "collègue",5),
    ("alice","claire","amie", 8),
    ("bob", "david", "collègue",3),
],
["src","dst","relation","force"])
```

```
g = GraphFrame(vertices, edges)
print(g.vertices.count(), g.edges.count())
```

Interroger un graphe

Requêtes directes sur les DataFrames

Degrés des sommets

```
g.degrees.orderBy(F.col('degree').desc()).show()  
g.inDegrees.show()  
g.outDegrees.show()
```

Filtrer les arêtes

```
g.edges.filter(  
    (F.col('src')== 'alice') &  
    (F.col('relation')== 'collègue')  
).show()
```

Relations de force > 7

```
g.edges.filter(F.col('force')>7).show()
```

Sommets de plus de 30 ans

```
g.vertices.filter(F.col('age')>30).show()
```

Checkpoint

Checkpoint obligatoire : plusieurs algorithmes GraphFrames (connectedComponents, SCC...) nécessitent un répertoire de checkpoint.

Toujours appeler `spark.sparkContext.setCheckpointDir( '/tmp/gf-checkpoints' )` avant de lancer ces algorithmes.

Recherche de motifs (Motif Finding)

Motif Finding

Syntaxe déclarative style Cypher

Trouver des structures récurrentes sans écrire de boucle — Spark génère le plan SQL optimal

Syntaxe : (sommet)-[arête]->(sommet) — noms = variables de capture

Tous les chemins $A \rightarrow B \rightarrow C$

```
chemins = g.find("(a)-[e1]->(b); (b)-[e2]->(c)")
chemins.select('a.id','b.id','c.id','e1.relation','e2.relation').show()
```

Relations réciproques ($A \rightarrow B$ ET $B \rightarrow A$)

```
reciproques = g.find("(a)-[e1]->(b); (b)-[e2]->(a)")
reciproques.select('a.id','b.id').show()
```

Triangles dirigés ($A \rightarrow B \rightarrow C \rightarrow A$)

```
triangles = g.find("(a)-[e1]->(b); (b)-[e2]->(c); (c)-[e3]->(a)")
triangles.select('a.id','b.id','c.id').show()
```

Amis d'amis (avec filtres supplémentaires)

```
amis_amis = g.find("(a)-[e1]->(b); (b)-[e2]->(c)")\
    .filter("e1.relation='ami' AND e2.relation='ami'")\
    .select('a.id','b.id','c.id').show()
```

Motif Finding

Exemples

Tous les chemins $A \rightarrow B \rightarrow C$

```
chemins = g.find("(a)-[e1]->(b); (b)-[e2]->(c)")
chemins.select('a.id','b.id','c.id','e1.relation','e2.relation').show()
```

Relations réciproques ($A \rightarrow B$ ET $B \rightarrow A$)

```
reciproques = g.find("(a)-[e1]->(b); (b)-[e2]->(a)")
reciproques.select('a.id','b.id').show()
```

Triangles dirigés ($A \rightarrow B \rightarrow C \rightarrow A$)

```
triangles = g.find("(a)-[e1]->(b); (b)-[e2]->(c); (c)-[e3]->(a)")
triangles.select('a.id','b.id','c.id').show()
```

Amis d'amis (avec filtres supplémentaires)

```
amis_amis = g.find("(a)-[e1]->(b); (b)-[e2]->(c)")\
    .filter("e1.relation='ami' AND e2.relation='ami'")\
    .filter('a.id != c.id')\
    .select('a.id','c.id').distinct()
```

Détection de fraude : compte offshore à 2 sauts d'un particulier

```
g.find("(p)-[]->(i); (i)-[]->(o)")\
    .filter("p.type='particulier' AND o.type='offshore'")
```

Motif Finding

Syntaxe déclarative style Cypher

Puissance de la syntaxe déclarative : `g.find()` traduit le motif en une jointure SQL optimisée par Catalyst.

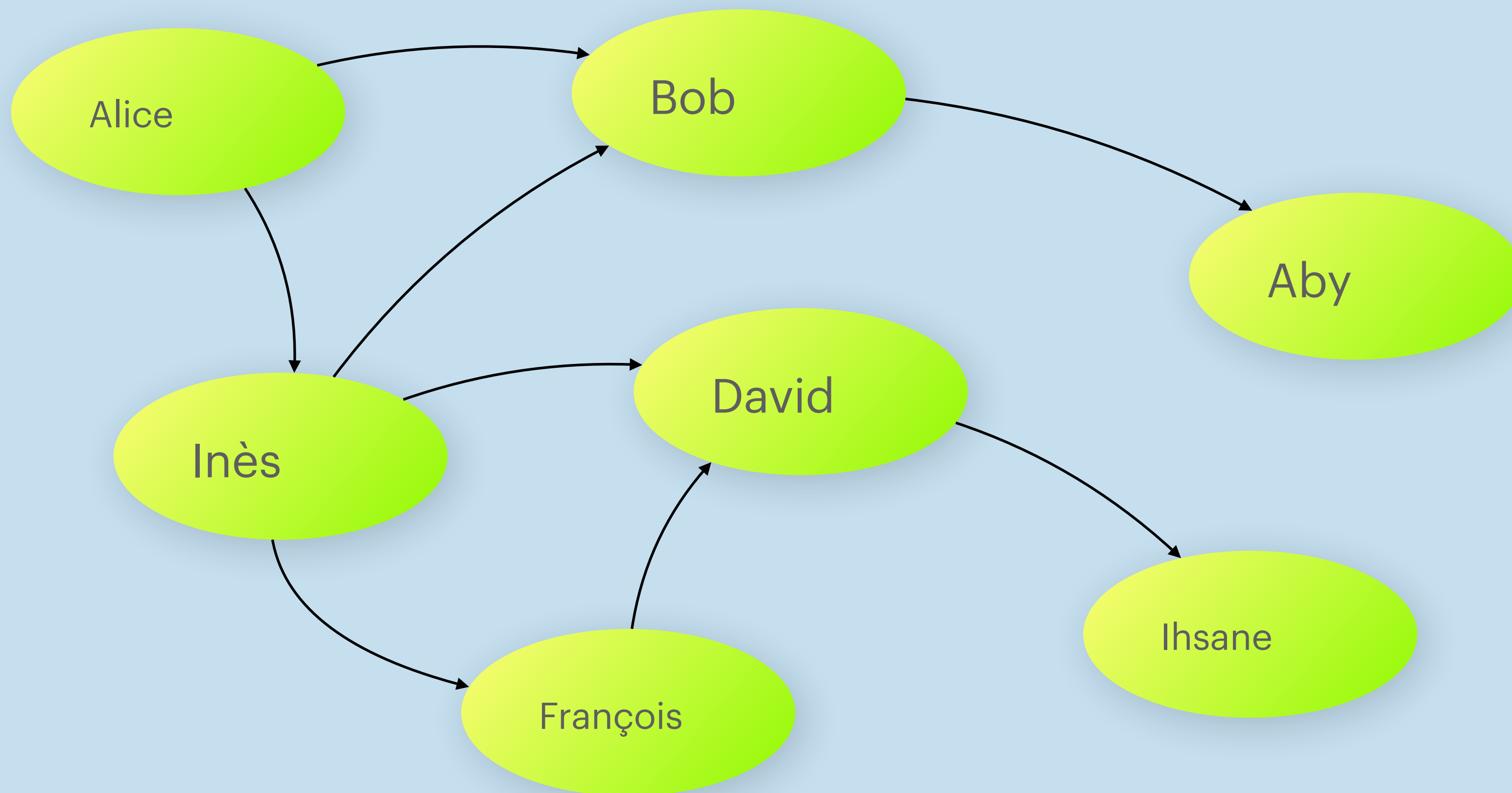
Plus le motif est complexe, plus cette approche est avantageuse par rapport à une implémentation manuelle en boucles.

Breadth-First Search

Recherche en largeur d'abord

BFS — Plus court chemin en nombre d'arêtes

Explore niveau par niveau depuis le
sommet source jusqu'au sommet cible



BFS — Plus court chemin en nombre d'arêtes

BFS depuis Alice vers François :

```
# BFS avec GraphFrames
resultat = g.bfs(
    fromExpr    = "id = 'alice'",
    toExpr      = "id = 'francois'",
    edgeFilter  = "relation != 'ennemi'", # Filtrer des arêtes (optionnel)
    maxPathLength = 5
)
resultat.show(truncate=False)

# Résultat : from | e0 | v1 | e1 | v2 | ... | to
# Colonnes alternées : sommets intermédiaires (v_i) et arêtes empruntées (e_i)
```

Composantes connexes

Composantes connexes et fortement connexes

Sous-graphes isolés —
incontournables pour la détection de
fraude et l'analyse de réseaux

Algorithme	Graphe	Cas d'usage typique
connectedComponents()	Non orienté	Détecter des îlots isolés, segments de clientèle déconnectés
stronglyConnectedComponents()	Orienté	Détecter des cycles de virements, boucles de dépendances

Composantes connexes et fortement connexes

Sous-graphes isolés — incontournables pour la détection de fraude et l'analyse de réseaux
Composantes connexes (non orienté)

```
# Checkpoint obligatoire avant connectedComponents
spark.sparkContext.setCheckpointDir('/tmp/gf')

# Composantes connexes
cc = g.connectedComponents()
cc.groupBy('component')\
  .agg(F.count('*').alias('taille'),
       F.collect_list('id').alias('membres'))\
  .orderBy(F.col('taille').desc()).show()

# Composantes FORTEMENT connexes (graphe orienté)
scc = g.stronglyConnectedComponents(maxIter=10)
scc.select('id','component').show()
```

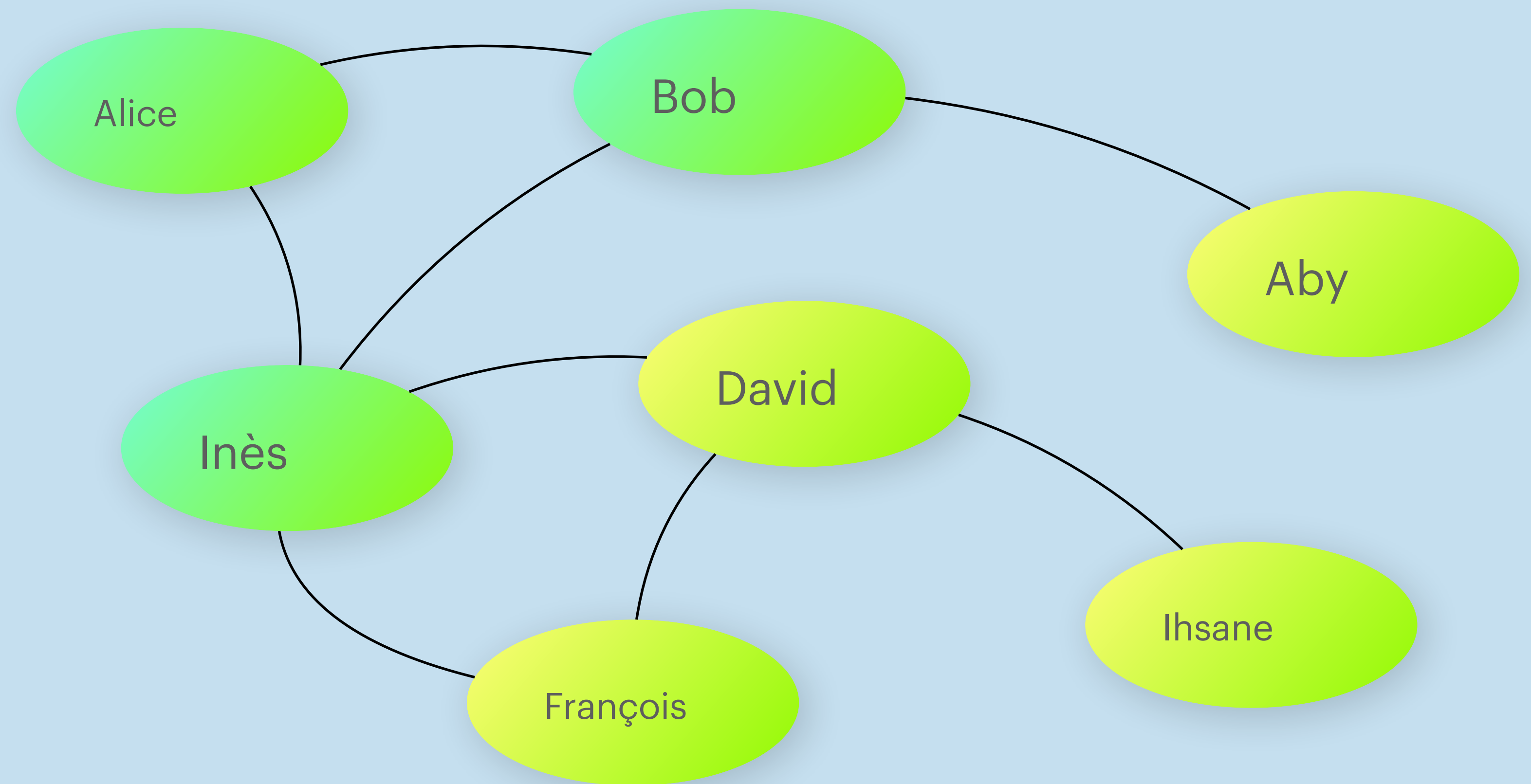
Comptage de triangles

Triangles

ou cycles de longueur 3

Triangle = 3 sommets
tous reliés deux à deux

Exemple :
[Alice, Bob, Inès]
forment un triangle



Coefficient de clustering

Mesure à quel point les voisins d'un sommet sont eux-mêmes interconnectés

$$C(v) = \frac{\text{triangles passant par } v}{\text{paires de voisins possibles de } v} = \frac{\text{triangles}(v)}{\deg(v) \cdot (\deg(v) - 1)/2}$$

Coefficient de clustering moyen : mesure la densité locale du graphe. Un réseau social aura un coefficient élevé (les amis sont souvent amis entre eux). Un réseau de citation scientifique aura un coefficient plus faible.

Comptage de triangles et coefficient de clustering

```
# Comptage de triangles par sommet
tc = g.triangleCount()
tc.select('id','count')\
  .orderBy(F.col('count').desc()).show()

# Coefficient de clustering local
deg_df = g.degrees
clustering = tc.join(deg_df,'id')\
  .withColumn('coeff',
    F.when(F.col('degree')<=1, 0.0)
    .otherwise(F.round(
      F.col('count')*2 /
      (F.col('degree')*(F.col('degree')-1)),4)))
clustering.orderBy(F.col('coeff').desc()).show()
```

PageRank

PageRank — Principe et formule mathématique

Intuition : une page est importante si de nombreuses pages importantes pointent vers elle. C'est une définition récursive résolue par itération — le PageRank modélise un surfeur aléatoire qui suit des liens avec probabilité **d** et se téléporte aléatoirement avec probabilité **1-d**.

$$PR(v) = (1 - d) + d \times \sum_{u \rightarrow v} PR(u) / \text{deg_out}(u)$$

d = facteur d'amortissement (damping factor) ≈ 0.85

u → v = tous les sommets u ayant une arête vers v

deg_out(u) = degré sortant de u — nb d'arêtes partant de u

Étape 0 — Init

$$PR(v) = 1/N \text{ pour tout sommet } v$$

Étape 1 — Itération t+1

$$PR_{\{t+1\}}(v) = (1-d)/N + d \times \sum PR_t(u) / \text{deg_out}(u)$$

Étape 2 — Convergence

$$\text{Répéter jusqu'à } |PR_{\{t+1\}} - PR_t| < \varepsilon \text{ (tolérance)}$$

resetProbability = **1 - d** = probabilité de téléportation. La valeur par défaut dans GraphFrames est 0.15 (d = 0.85). Réduire resetProbability donne plus de poids aux liens — augmenter le donne plus de poids à la téléportation aléatoire.

PageRank — Principe et formule mathématique

Page & Brin à Stanford (1998) — fondement de Google Search

Intuition : une page est importante si de nombreuses pages importantes pointent vers elle. C'est une définition récursive résolue par itération — le PageRank modélise un surfeur aléatoire qui suit des liens avec probabilité **d** et se téléporte aléatoirement avec probabilité **1-d**.

Étape 0 — Init

$$PR(v) = \frac{1}{N} \quad \text{pour tout sommet } v$$

Étape 1 — Itération t+1

$$PR_{t+1}(v) = \frac{1-d}{N} + d \times \sum \frac{PR_t(u)}{\text{deg}_{\text{out}}(u)}$$

Étape 2 — Convergence

Répéter jusqu'à $\|PR_{t+1} - PR_t\| < \varepsilon$ (tolérance)

resetProbability = 1 - d = probabilité de téléportation. La valeur par défaut dans GraphFrames est 0.15 (d = 0.85). Réduire resetProbability donne plus de poids aux liens — augmenter le donne plus de poids à la téléportation aléatoire.

PageRank avec GraphFrames et implémentation manuelle

Cas d'usage PageRank : moteur de recherche web (pages les plus autoritaires) · détection d'articles scientifiques influents · identification des utilisateurs les plus influents dans un réseau social · scoring de comptes centraux dans la détection de fraude.

PageRank personnalisé

La téléportation cible un sous-ensemble de sommets → mesure la proximité relative

PageRank pondéré

Les contributions sont pondérées par le poids des arêtes (force de la relation)

Implémentation avec GraphFrames

Méthode 1 : convergence auto (recommandée)

```
pr = g.pageRank(  
    resetProbability=0.15, # 1 - d  
    tol=0.001              # tolérance  
)  
pr.vertices.select('id','nom','pagerank')\  
    .orderBy(F.col('pagerank').desc()).show()
```

Méthode 2 : nombre fixe d'itérations

```
pr = g.pageRank(resetProbability=0.15, maxIter=20)
```

Les poids des arêtes sont aussi mis à jour :

```
pr.edges.select('src','dst','weight').show()
```

```
def pagerank_manuel(g, d=0.85, n_iter=20) :
```

```
    n = g.vertices.count()  
    pr = g.vertices.select('id')\  
        .withColumn('pr',F.lit(1.0/n))  
    out = g.outDegrees  
    for _ in range(n_iter):  
        contrib = g.edges\  
            .join(pr.withColumnRenamed('id','src')  
                .withColumnRenamed('pr','pr_s'),'src')\  
            .join(out.withColumnRenamed('id','src')  
                .withColumnRenamed('outDegree','deg'),'src')\  
            .withColumn('c',F.col('pr_s')/F.col('deg'))\  
            .groupBy('dst').agg(F.sum('c').alias('sc'))\  
            .withColumnRenamed('dst','id')  
        pr = g.vertices.select('id')\  
            .join(contrib,'id','left')\  
            .withColumn('pr',(1-d)/  
n+d * F.coalesce('sc',F.lit(0)))\  
            .select('id','pr')  
    return pr
```

Propagation de labels et plus courts chemins

Label Propagation et Shortest Paths

Label Propagation — détection de communautés non supervisées

Shortest Paths — distances depuis des landmarks

```
# Label Propagation
Init : chaque sommet a son propre label (son id)
lpa = g.labelPropagation(maxIter=10)
Iterations : chaque sommet adopte le label majoritaire de ses voisins
Résultat : groupes stables = communautés
lpa.select('id', 'nom', 'label').orderBy('label').show()
```

```
# Visualiser les communautés

lpa.groupBy('label').agg(
    F.count('*').alias('taille'),
```

```
# Distances depuis un ensemble de landmarks
landmarks = ['alice', 'francois']
sp = g.shortestPaths(landmarks=landmarks)
sp.select('id','distances').show(truncate=False)
```

```
# Résultat :
# id      | distances
# alice   | {alice→0, francois→2}
# bob     | {alice→1, francois→3}
# claire  | {alice→1, francois→1}

# Filtrer les sommets à distance 1 d'alice
sp.filter("distances['alice']=1").show()
```

Algorithme	Méthode GraphFrames	Cas d'usage
Label Propagation	g.labelPropagation(maxIter)	Communautés, segmentation, détection de thèmes
Shortest Paths	g.shortestPaths(landmarks)	Distance à des nœuds clés, centralité
BFS	g.bfs(fromExpr, toExpr)	Chemin le plus court entre 2 sommets
Triangle Count	g.triangleCount()	Coefficient de clustering, densité locale

Propagation de labels

Détection de communautés non supervisée

Algorithme :

- 1) Initialiser les labels de tous les nœuds du réseau.
Pour un nœud donné x , $C_x(0) = x$
- 2) Poser $t = 1$
- 3) Organiser les nœuds du réseau dans un ordre aléatoire et les stocker dans X .
- 4) Pour chaque $x \in X$, suivi dans cet ordre spécifique :
$$C_x(t) = f\left(C_{x_{i_1}}(t), \dots, C_{x_{i_m}}(t), C_{x_{i_{m+1}}}(t-1), \dots, C_{x_{i_k}}(t-1)\right)$$
- 5) Si **chaque nœud** possède un label que la **majorité de ses voisins** possède également, alors **arrêter l'algorithme** ; Dans le cas contraire, poser $t=t+1$ et retourner à l'étape **(3)**.

Label Propagation

```
lpa = g.labelPropagation(maxIter=10)
lpa.select('id','nom','label').orderBy('label').show()
```

Visualiser les communautés

```
lpa.groupBy('label').agg(
    F.count('*').alias('taille'),
    F.collect_list('nom').alias('membres')
).orderBy(F.col('taille').desc()).show(truncate=False)
```

Plus courts chemins

Distances depuis des « landmarks »

```
# Distances depuis un ensemble de landmarks
landmarks = ['alice', 'francois']
sp = g.shortestPaths(landmarks=landmarks)
sp.select('id','distances').show(truncate=False)
```

```
# Résultat :
```

```
# id      | distances
# alice   | {alice→0, francois→2}
# bob     | {alice→1, francois→3}
# claire  | {alice→1, francois→1}
```

```
# Filtrer les sommets à distance 1 d'alice
sp.filter("distances['alice'] = 1").show()
```

Résumé

Concept	Points clés à retenir
Graphe $G = (V, E)$	Sommets + arêtes — orienté ou non, pondéré ou non — $G = (V, E)$
GraphFrames	API Python sur DataFrames — vertices (col. id) + edges (col. src, dst)
Checkpoint	Obligatoire pour connectedComponents et SCC — toujours configurer avant
Motif Finding	Syntaxe (a)-[e]->(b) — recherche déclarative de structures complexes
BFS	Plus court chemin en nb d'arêtes — <code>g.bfs(fromExpr, toExpr, maxPathLength)</code>
Composantes connexes	Sous-graphes isolés — <code>g.connectedComponents()</code> / <code>stronglyConnectedComponents()</code>
Triangle Count	Mesure de transitivité locale — base du coefficient de clustering
PageRank	Score d'importance récursif — <code>g.pageRank(resetProbability=0.15, tol=0.001)</code>
Damping factor d	$d = 0.85$ (défaut) — <code>resetProbability = 1 - d</code> = probabilité de téléportation
Label Propagation	Détection de communautés non supervisée — <code>g.labelPropagation(maxIter)</code>
Shortest Paths	Distances depuis des landmarks — <code>g.shortestPaths(landmarks=[...])</code>

Exercices

Ex. 1 — Modélisation — réseau ferroviaire (20 min) Sommets : gares (nom, ville, nb_quais). Arêtes : liaisons (durée_min, distance_km, compagnie). Créer le GraphFrame, trouver la gare la mieux connectée, lister les gares accessibles depuis Paris en ≤ 2 étapes.

Ex. 2 — Réseau social aléatoire (35 min) 50 utilisateurs, 200 relations. Calculer les degrés entrant/sortant — top 5 influents. Détecter les composantes connexes. Calculer le coefficient de clustering moyen. Appliquer Label Propagation.

Ex. 3 — PageRank pédagogique (40 min) Sur 10 sommets et 25 arêtes : implémenter PageRank manuellement (boucle itérative). Comparer avec `g.pageRank()`. Faire varier `d` entre 0.5 et 0.95. Observer l'impact des dangling nodes (sommets sans arête sortante).

Ex. 4 — Détection de fraude bancaire (45 min) Sommets : comptes (particulier, entreprise, offshore). Arêtes : virements (montant, date). BFS pour trouver les comptes offshore à ≤ 2 sauts d'un particulier. Cycles de longueur 3 via motifs. PageRank des comptes les plus centraux.

Pour aller plus loin

Documentation et outils

- * GraphFrames :
graphframes.github.io/graphframes/docs/_site/user-guide.html
- * GraphX (Scala) :
spark.apache.org/docs/latest/graphx-programming-guide.html
- * NetworkX : bibliothèque Python de graphes pour prototyper avant de passer à GraphFrames
networkx.org
- * Neo4j : base de données graphe avec Cypher — visualiser et explorer les graphes
neo4j.com
- * SNAP (Stanford) : jeux de données graphes réels pour les TPs
nap.stanford.edu/data

Articles fondateurs

Page, Brin et al. — *The PageRank Citation Ranking: Bringing Order to the Web*

Stanford Technical Report, 1998 — l'article fondateur de PageRank et de Google

Raghavan et al. — *Near Linear Time Algorithm to Detect Community Structures in Large-Scale Networks*

Physical Review E, 2007 — fondement de l'algorithme Label Propagation